

SHETH L.U.J & SIR MV COLLEGE

Practical 1: Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

#GRAPH

```
website_graph = {  
    'Home': ['Login', 'Products', 'Contact'],  
    'Login': [],  
    'Products': ['Electronics', 'Clothing'],  
    'Electronics': [],  
    'Clothing': [],  
    'Contact': []  
}
```

#BFS

```
from collections import deque  
def bfs_website_navigation(graph, start_node):  
    visited = set()  
    queue = deque([start_node])  
    traversal_order = []  
    while queue:  
        current_page = queue.popleft()  
        if current_page not in visited:  
            visited.add(current_page)  
            traversal_order.append(current_page)  
            for neighbor in graph[current_page]:  
                if neighbor not in visited:  
                    queue.append(neighbor)  
    return traversal_order  
bfs_result = bfs_website_navigation(website_graph, 'Home')  
print(f"BFS Traversal Order: {bfs_result}")
```

#OUTPUT

PURVI.R.KARKERA
T086

SHETH L.U.J & SIR MV COLLEGE

The screenshot shows a Google Colab interface with a notebook titled "T086-Practical 1: Breadth First Search & Iterative Depth First Search in Python.ipynb". The code in the notebook includes a BFS function and its execution. The output of the BFS function is displayed as a list: `['Home', 'Login', 'Products', 'Contact', 'Electronics', 'Clothing']`. The interface also shows a terminal at the bottom and various Colab controls like 'Run all', 'Share', and 'RAM/Disk' usage.

#DFS

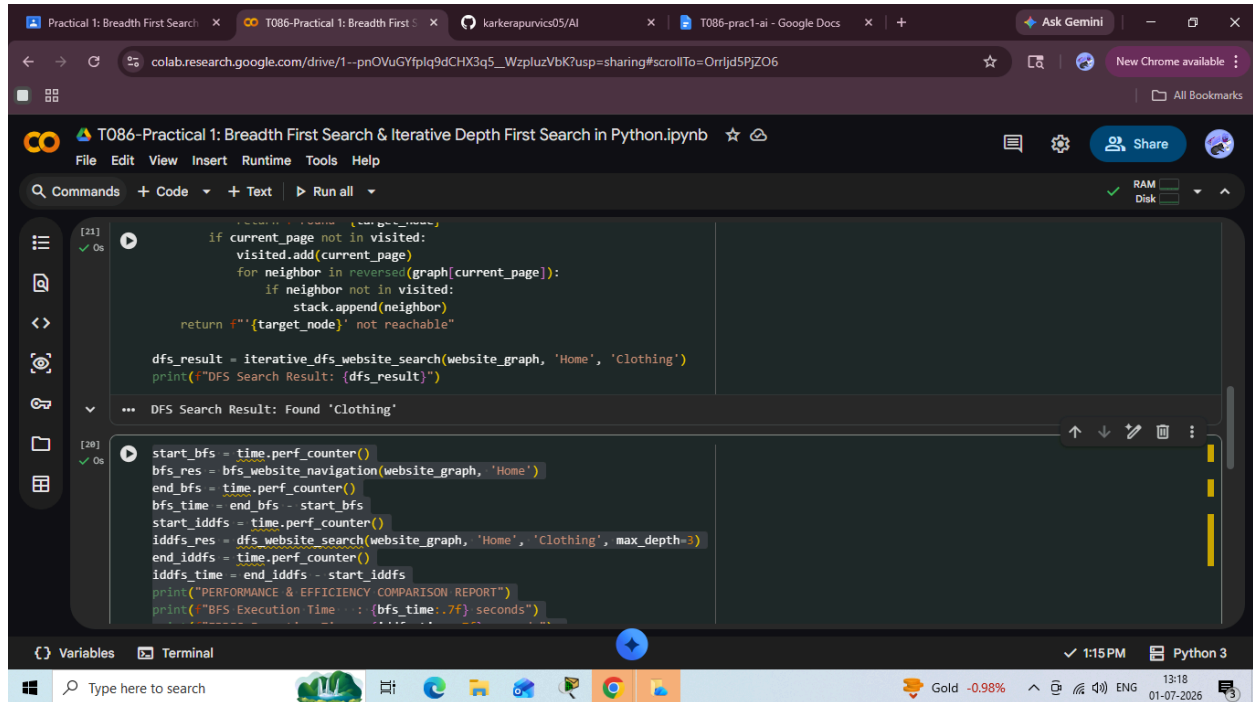
```
def iterative_dfs_website_search(graph, start_node, target_node):
    visited = set()
    stack = [start_node]
    while stack:
        current_page = stack.pop()
        if current_page == target_node:
            return f"Found '{target_node}'"
        if current_page not in visited:
            visited.add(current_page)
            for neighbor in reversed(graph[current_page]):
                if neighbor not in visited:
                    stack.append(neighbor)
    return f"'{target_node}' not reachable"

dfs_result = iterative_dfs_website_search(website_graph, 'Home',
                                          'Clothing')
print(f"DFS Search Result: {dfs_result}")
```

#OUTPUT

PURVI.R.KARKERA
T086

SHETH L.U.J & SIR MV COLLEGE



```
[21] ✓ 0s
    if current_page not in visited:
        visited.add(current_page)
        for neighbor in reversed(graph[current_page]):
            if neighbor not in visited:
                stack.append(neighbor)
        return f"'{target_node}' not reachable"

    dfs_result = iterative_dfs_website_search(website_graph, 'Home', 'Clothing')
    print(f"DFS Search Result: {dfs_result}")

... DFS Search Result: Found 'Clothing'

[20] ✓ 0s
start_bfs = time.perf_counter()
bfs_res = bfs_website_navigation(website_graph, 'Home')
end_bfs = time.perf_counter()
bfs_time = end_bfs - start_bfs
start_iddfs = time.perf_counter()
iddfs_res = dfs_website_search(website_graph, 'Home', 'Clothing', max_depth=3)
end_iddfs = time.perf_counter()
iddfs_time = end_iddfs - start_iddfs
print("PERFORMANCE & EFFICIENCY COMPARISON REPORT")
print(f"BFS Execution Time : {bfs_time:.7f} seconds")
print(f"IDDFS Execution Time : {iddfs_time:.7f} seconds")
print("Memory Efficiency Context:")
print("- BFS Space Complexity : O(b^d) -> High memory (stores full levels in queue)")
print("- IDDFS Space Complexity : O(d) -> Low memory (stores only current path)")
print("Operational Behavior Summary:")
print("- BFS visits broad layers sequentially, ideal for finding shortest path quickly.")
```

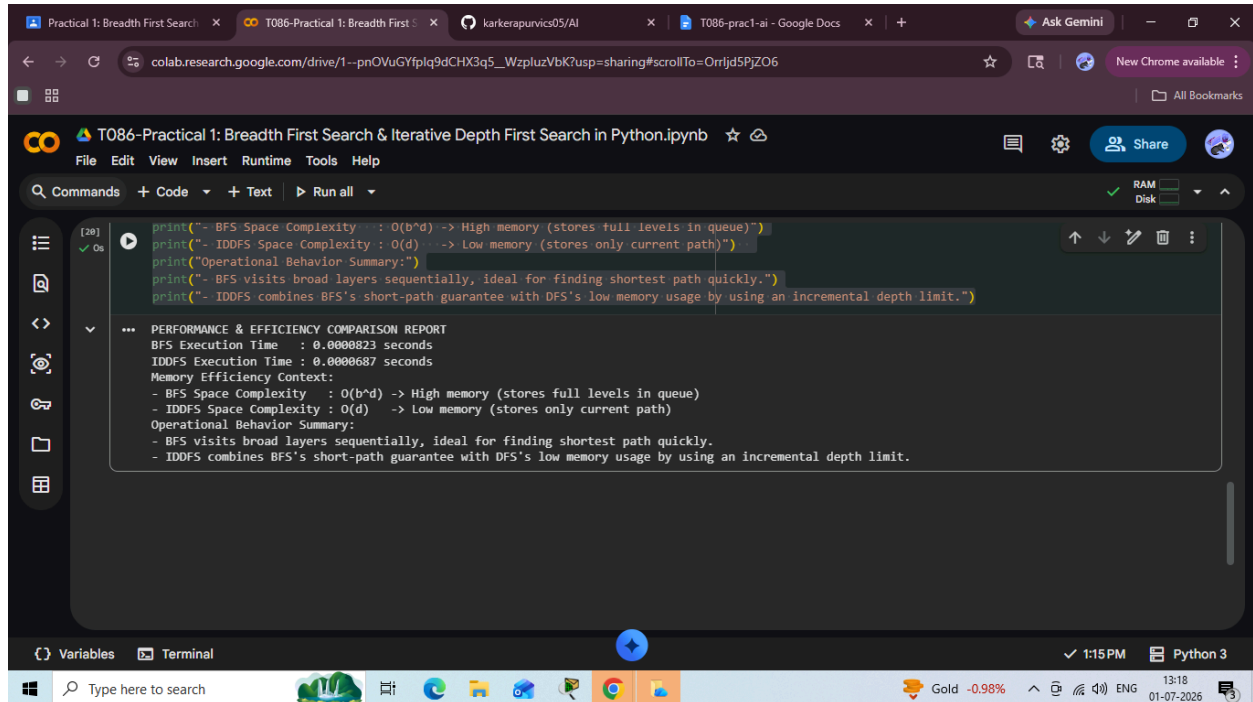
#COMPARISION

```
start_bfs = time.perf_counter()
bfs_res = bfs_website_navigation(website_graph, 'Home')
end_bfs = time.perf_counter()
bfs_time = end_bfs - start_bfs
start_iddfs = time.perf_counter()
iddfs_res = dfs_website_search(website_graph, 'Home', 'Clothing',
max_depth=3)
end_iddfs = time.perf_counter()
iddfs_time = end_iddfs - start_iddfs
print("PERFORMANCE & EFFICIENCY COMPARISON REPORT")
print(f"BFS Execution Time : {bfs_time:.7f} seconds")
print(f"IDDFS Execution Time : {iddfs_time:.7f} seconds")
print("Memory Efficiency Context:")
print("- BFS Space Complexity : O(b^d) -> High memory (stores full levels in queue)")
print("- IDDFS Space Complexity : O(d) -> Low memory (stores only current path)")
print("Operational Behavior Summary:")
print("- BFS visits broad layers sequentially, ideal for finding shortest path quickly.")
```

SHETH L.U.J & SIR MV COLLEGE

```
print("- IDDFS combines BFS's short-path guarantee with DFS's low memory usage by using an incremental depth limit.")
```

#OUTPUT



The screenshot shows a Google Colab notebook titled "T086-Practical 1: Breadth First Search & Iterative Depth First Search in Python.ipynb". The code cell contains the following Python code:

```
print("- BFS Space Complexity : O(b*d) -> High memory (stores full levels in queue)")
print("- IDDFS Space Complexity : O(d) -> Low memory (stores only current path)")
print("Operational Behavior Summary:")
print("- BFS visits broad layers sequentially, ideal for finding shortest path quickly.")
print("- IDDFS combines BFS's short-path guarantee with DFS's low memory usage by using an incremental depth limit.")
```

The output of the code is a performance report:

```
... PERFORMANCE & EFFICIENCY COMPARISON REPORT
BFS Execution Time : 0.0000823 seconds
IDDFS Execution Time : 0.0000687 seconds
Memory Efficiency Context:
- BFS Space Complexity : O(b*d) -> High memory (stores full levels in queue)
- IDDFS Space Complexity : O(d) -> Low memory (stores only current path)
Operational Behavior Summary:
- BFS visits broad layers sequentially, ideal for finding shortest path quickly.
- IDDFS combines BFS's short-path guarantee with DFS's low memory usage by using an incremental depth limit.
```

The notebook interface shows the code is executed successfully, with a RAM and Disk usage indicator at the top right. The bottom status bar indicates the time is 1:15 PM and the environment is Python 3.